In-Depth Usage Guide for `gum`

`gum` is a command-line tool that brings the power of the Charm TUI (Terminal User Interface) libraries to your shell scripts. It allows you to create beautiful, interactive, and user-friendly command-line interfaces with minimal effort, making your scripts more engaging and easier to use.

1. Installation

`gum` is typically installed via a package manager or by downloading the pre-compiled binaries.

Using Homebrew (macOS/Linux):

```
brew install gum
```

Using Go (if you have Go installed):

```
go install github.com/charmbracelet/gum@latest
```

Manual Installation: You can also download the appropriate binary for your system from the Charm Bracelet releases page on GitHub, extract it, and place it in your system's `PATH` (e.g., `/usr/local/bin`).

2. Core Commands and Examples

`gum` provides a set of composable commands, each designed for a specific UI element or interaction.

2.1 `gum input` - Get User Input

Prompts the user for text input.

Basic Usage:

```
name=$(gum input --placeholder "What's your name?")
echo "Hello, $name!"
```

With a default value:

```
project_name=$(gum input --placeholder "Project Name" --value
"MyAwesomeProject")
echo "Creating project: $project_name"
```

Password input (hidden characters):

```
password=$(gum input --password --placeholder "Enter your password")
echo "Password entered (hidden): $password"
```

2.2 `gum confirm` - Get Yes/No Confirmation

Asks the user for a yes/no confirmation. Exits with status 0 for “yes” and 1 for “no”.

Basic Usage:

```
if gum confirm "Are you sure you want to proceed?"; then
    echo "Proceeding..."
else
    echo "Aborting."
fi
```

Customizing prompts and default:

```
if gum confirm --affirmative="Go for it!" --negative="No way!" --
default=false "This action is irreversible. Continue?"; then
    echo "Action initiated."
else
    echo "Action cancelled."
fi
```

2.3 `gum choose` - Select from a List

Allows the user to select one or more items from a list.

Single selection:

```
fruit=$(gum choose "Apple" "Banana" "Orange" "Grape")
echo "You chose: $fruit"
```

Multiple selections:

```
toppings=$(gum choose --limit 2 "Pepperoni" "Mushrooms" "Onions"
"Olives" "Extra Cheese")
echo "Selected toppings: $toppings" # Output will be space-separated
```

Reading options from stdin:

```
ls -1 | gum choose --limit 1 --height 10 "Select a file:"
```

2.4 `gum filter` - Fuzzy Search a List

Filters a list of items using fuzzy matching, similar to `fzf`.

Basic Usage:

```
selected_file=$(ls -1 | gum filter --placeholder "Search for a
file...")
echo "Selected: $selected_file"
```

With a custom prompt:

```
command_to_run=$(history | gum filter --prompt "Run a past
command:")
echo "Running: $command_to_run"
```

2.5 `gum format` - Format Text with Markdown/Emoji

Formats text using Markdown, emoji, and syntax highlighting.

Markdown formatting:

```
gum format "# Hello, Gum!

 This is **bold** and *italic* text."
```

Code highlighting:

```
gum format '```python
print("Hello, World!")
```'
```

**Emoji support:**

```
gum format "✨ This is a sparkling message! ✨"
```

2.6 `gum spin` - Show a Spinner While Running a Command

Displays a spinner while a command is executing, providing visual feedback.

Basic Usage:

```
gum spin --title "Processing..." -- sleep 3 && echo "Done!"
```

Customizing spinner and text:

```
gum spin --spinner dot --title "Downloading updates..." -- curl -s -o /dev/null https://example.com
echo "Updates downloaded."
```

2.7 `gum write` - Multi-line Text Input

Allows the user to input multi-line text, useful for descriptions or longer messages.

Basic Usage:

```
description=$(gum write --placeholder "Enter a detailed description (Ctrl+D to finish):")
echo "Description:
$description"
```

With a pre-filled value:

```
notes=$(gum write --value "Initial notes:
- Item 1
- Item 2" --height 10 "Add more notes:")
echo "Final notes:
$notes"
```

2.8 `gum style` - Apply Styles to Text

Applies ANSI styles (colors, bold, italic, etc.) to text.

Basic Usage:

```
gum style --foreground "#FF0000" --bold "This text is red and bold."
```
```

Multiple styles:

```
gum style --foreground "green" --italic --underline "Success! This
is a styled message."
```

Using a block for styling:

```
gum style \
    --border double \
    --border-foreground 212 \
    --align center \
    --width 50 \
    --margin "1 2" \
    --padding "2 4" \
    "Welcome to my script!"
```

2.9 `gum join` - Join Text Vertically or Horizontally

Combines multiple strings or outputs, either vertically or horizontally.

Vertical join (default):

```
gum join "Line 1" "Line 2" "Line 3"
```

Horizontal join:

```
gum join --horizontal "Column A" "Column B" "Column C"
```

Joining command outputs:

```
gum join --horizontal \
    <(echo "User:") \
    <(whoami) \
    <(echo "Current Directory:") \
    <(pwd)
```

2.10 `gum file` - Select a File or Directory

Opens a file picker for selecting files or directories.

Select a file:

```
selected_file=$(gum file --file)
echo "Selected file: $selected_file"
```

Select a directory:

```
selected_dir=$(gum file --directory)
echo "Selected directory: $selected_dir"
```

Starting in a specific directory:

```
selected_path=$(gum file --file --path ~/Documents)
echo "Selected: $selected_path"
```

2.11 `gum pager` - View Long Content

Displays long content in a scrollable pager, similar to `less`.

Basic Usage:

```
cat /etc/fstab | gum pager
```

With a custom prompt:

```
long_log_output | gum pager --prompt "Press 'q' to quit"
```

2.12 `gum table` - Display Data in a Table

Presents data in a formatted table. Input is typically CSV or space-separated.

Basic Usage (space-separated):

```
echo "Name Age City
Alice 30 NewYork
Bob 25 London" | gum table
```

With headers and CSV input:

```
echo "Name,Age,City
Alice,30,New York
Bob,25,London" | gum table --csv --headers
```

3. Styling and Customization

Most `gum` commands support a wide range of flags for styling and customization. These often include:

- `--foreground`, `--background`: Set text and background colors (e.g., `red`, `#FF0000`, `212`).
- `--bold`, `--italic`, `--underline`, `--strikethrough`: Apply text decorations.
- `--border`, `--border-foreground`, `--border-background`: Control borders around elements.
- `--width`, `--height`: Set dimensions for UI elements.
- `--margin`, `--padding`: Control spacing.
- `--align`: Text alignment (e.g., `left`, `center`, `right`).
- `--prompt`: Customize the prompt text.
- `--selected-foreground`, `--selected-background`: For `choose` and `filter`, customize selected item appearance.

You can find a full list of flags for each command by running `gum <command> --help` (e.g., `gum input --help`).

4. Integrating with Shell Scripts

`gum` is designed to be easily integrated into any shell script. Its commands output their results to `stdout` and indicate success/failure via exit codes, making them perfect for use in `if` statements, variable assignments, and pipelines.

Example: A Simple Interactive Script

```
#!/bin/bash
```

```
        gum style \
            --border double \
            --border-foreground 212 \
            --align center \
            --width 60 \
            --margin "1 2" \
            --padding "2 4" \
            "Welcome to the Project Setup Wizard!"

        project_name=$(gum input --placeholder "Enter project name:")

        if [ -z "$project_name" ]; then
            gum style --foreground "red" "Project name cannot be empty.
Aborting."
            exit 1
        fi

        framework=$(gum choose "React" "Vue" "Angular" "Svelte" --header
"Choose a frontend framework:")

        if gum confirm "Do you want to initialize a Git repository?"; then
            init_git="yes"
        else
            init_git="no"
        fi

        gum spin --title "Processing..." -- \
            bash -c "mkdir $project_name && cd $project_name && sleep 2"

        if [ "$init_git" == "yes" ]; then
            gum spin --title "Initializing Git repository..." -- \
                bash -c "cd $project_name && git init && sleep 1"
        fi

        gum style --foreground "green" "Project '$project_name' created
successfully!"
        echo "Framework: $framework"
        echo "Git initialized: $init_git"
```

5. Advanced Usage and Tips

- **Piping and Redirection:** `gum` commands work seamlessly with pipes (`|`) for input and output redirection (`>`, `<`).
- **Error Handling:** Always check the exit status of `gum` commands, especially `gum confirm`, to handle user cancellations or errors gracefully.
- **Chaining Commands:** Combine multiple `gum` commands to build complex interactive flows. For example, use `gum input` to get a search query, then `gum filter` to narrow down results based on that query.
- **Dynamic Content:** Generate lists for `gum choose` or `gum filter` dynamically from other commands or scripts.
- **Theming:** While `gum` doesn't have a global theme file, you can create shell functions or aliases to apply consistent styling across your scripts.
- **Debugging:** If a `gum` command isn't behaving as expected, try running it with `set -x` in your bash script to see the exact commands being

executed and their outputs.